COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Jul 28, 2026	**DATE SUBMITTED:**	Aug 1, 2026
NAME:	Yazinel Corro	**INSTRUCTOR:**	Engr.Jim Lord Quejado

ACTIVITY NO. 4.1

Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

ILOA:

Input:

```
#include<stack> // calling stack from the STL

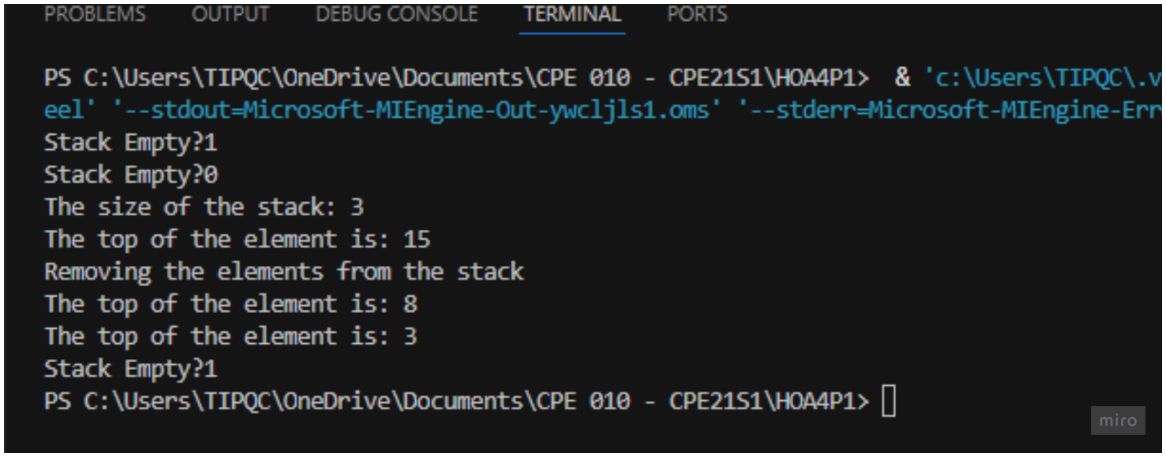
int main(){
// create an object for the stack
std::stack<int> newStack;

std::cout<<"Stack Empty?" <<newStack.empty()<<std::endl;
//push to the stack
newStack.push(3);
newStack.push(8);
newStack.push(15);

std::cout<<"Stack Empty?" <<newStack.empty()<<std::endl;
std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
std::cout<<"Removing the elements from the stack \n";
newStack.pop();
std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
newStack.pop();
std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
newStack.pop();
std::cout<<"Stack Empty?" <<newStack.empty()<<std::endl;


return 0;
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\TIPQC\OneDrive\Documents\CPE 010 - CPE21S1\HOA4P1> & 'c:\Users\TIPQC\.v
eel' '--stdout=Microsoft-MIEngine-Out-ywcljls1.oms' '--stderr=Microsoft-MIEngine-Err
Stack Empty?1
Stack Empty?0
The size of the stack: 3
The top of the element is: 15
Removing the elements from the stack
The top of the element is: 8
The top of the element is: 3
Stack Empty?1
PS C:\Users\TIPQC\OneDrive\Documents\CPE 010 - CPE21S1\HOA4P1>
miro

ILOB:

Input:

```cpp
#include<iostream>


const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;


void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void showAll();


int main(){
    int choice;
    std::cout<<"Enter the max elements for the new stack element: ";
    std::cin>>i;


    while(true){
        std::cout<<"- - - - - Stack Operations - - - - -"<<std::endl;
        std::cout<<"1. Push \n2. POP \n3. TOP \n4. isEMPTY \n5. Show All Values"<<std::endl;
        std::cin>> choice;


        switch (choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<<std::endl;
            break;
            case 5: showAll();
            break;
            default: std::cout<<"invalid choice."<< std::endl;
            break;
        }
    }
```

```cpp
int main(){



    return 0;


}


bool isEmpty(){
    if(top == -1) return true;
    return false;
}


bool isFull(){
    if(top > i-1) return true;
    return false;
}


void push(){
    if(isFull()){
        std::cout<<"Stack Overflow";
        return;
    }


    std::cout<<"New Value to add into the stack"<<std::endl;
    std::cin >> newData;
    stack[++top] = newData;
}

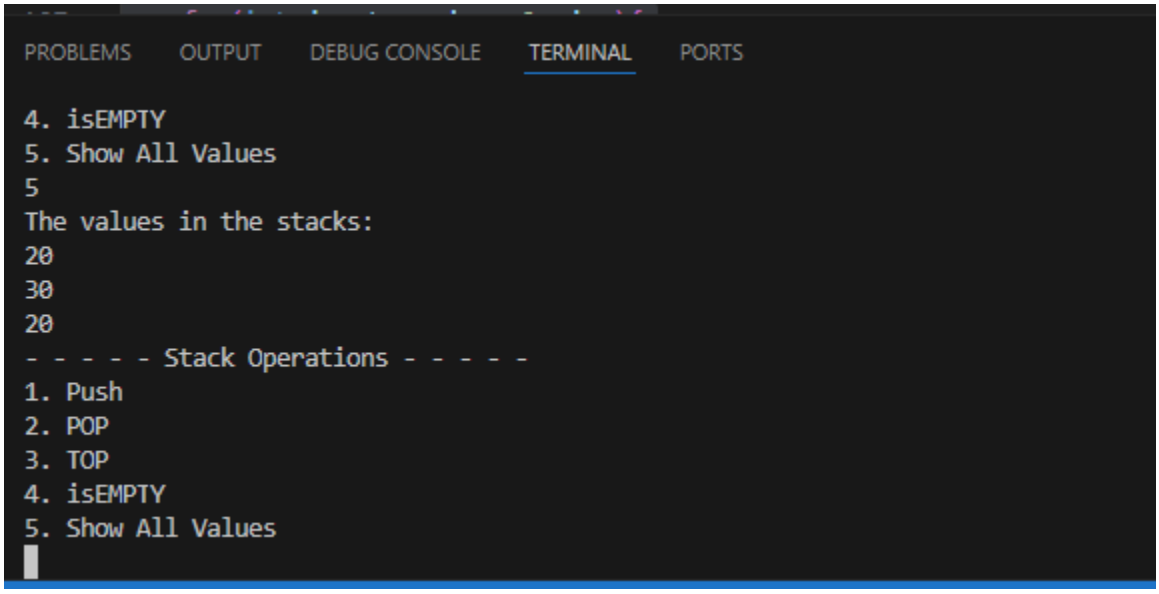

void Top(){
    if(isEmpty()){
        std::cout<<"Stack Underflow."<<std::endl;
    }


    std::cout<<"The element on top of the stack is "<<stack[top]<<std::endl;
}

```

```
66  void push(){
75      stack[++top] = newData;
76  }
77
78
79  void Top(){
80      if(isEmpty()){
81          std::cout<<"Stack Underflow."<<std::endl;
82      }
83
84
85      std::cout<<"The element on top of the stack is "<<stack[top]<<std::endl;
86  }
87
88
89  void pop(){
90      if(isEmpty()){
91          std::cout<<"Stack Underflow."<<std::endl;
92      }
93      std::cout<<"popping"<<std::endl;
94  }
95
96
97  void showAll(){
98      std::cout<<"The values in the stacks: " << std::endl;
99
100
101     if(isEmpty()){
102         std::cout << "Stack is empty." << std::endl;
103         return;
104     }
105
106
107     for(int j = top; j >= 0; j--){
108         std::cout << stack[j] << std::endl;
109     }
110 }
111
112
113
```

Output:



Observation:

- From the output, I observed that the stack stores all elements in a single array and follows the Last-In, First-Out principle. Every time a value is pushed, it is placed at the top of the stack, and when a value is popped, the most recently added element is removed first.

Conclusion:

In my conclusion, today's code was hard, but it was worth it because I learned many new concepts I still cannot keep up with every syntax, but I am doing my best to understand each part of the program step by step. Even though I encountered several errors while coding, I learned how to identify and fix them.

ILO C: Solve problems using an implementation of stack:

Code:
Int Main

```cpp
#include<iostream>
#include "stacklist.h"


int main(){


    std::cout<<"--- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";


    push('c');
    push('p');
    push('e');

    displayAll<char>();

    Top<char>();
    std::cout<<"\n ------ Popping all the elements ------ \n";

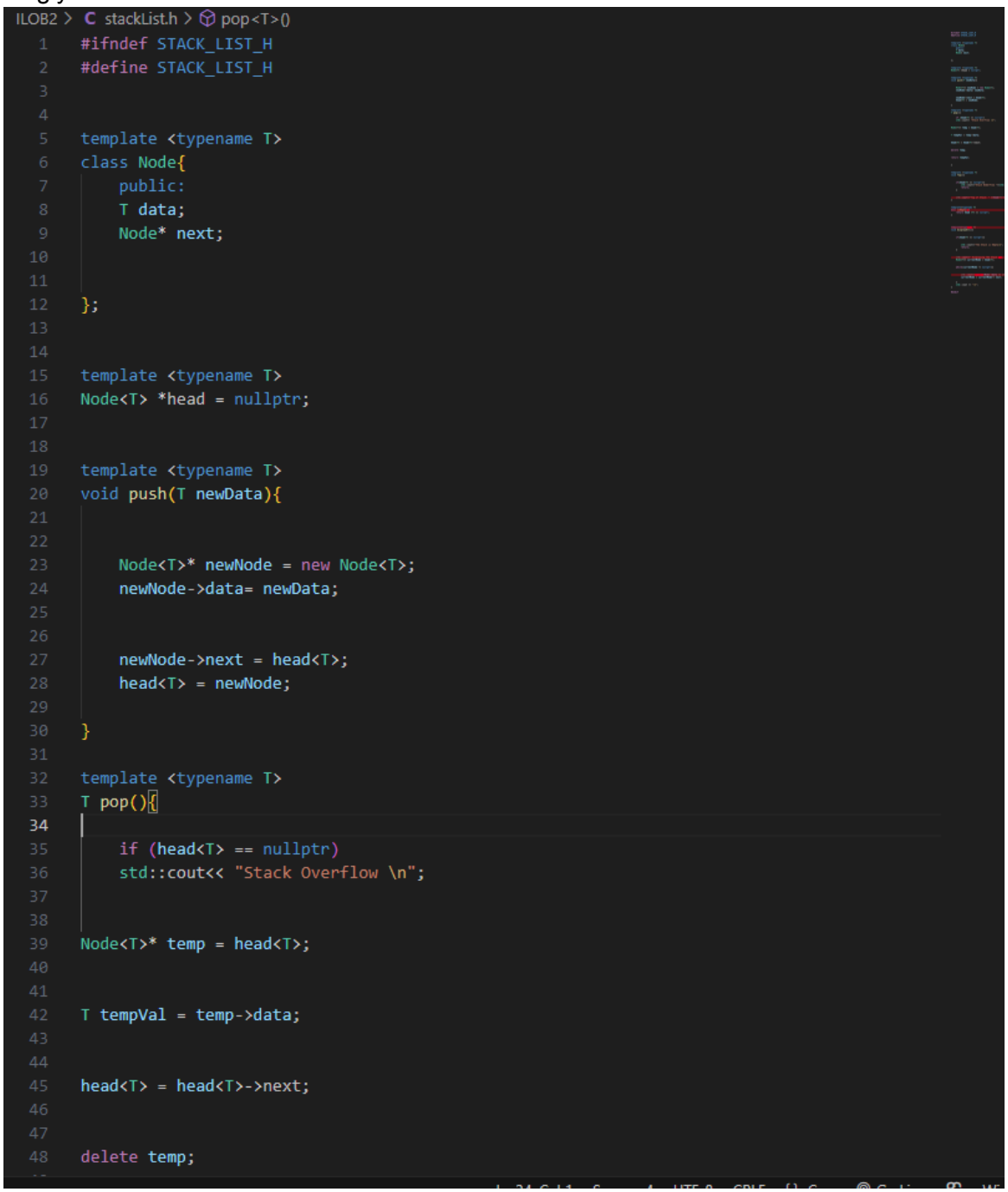
    while(!isEmpty<char>()){
        std::cout<<"Popped: "<< pop<char>() <<std::endl;
    }

    std::cout<<"\n ----- Testing the Underflow ----- \n";
    pop<char>();

    return 0;
}

```

Singly:


ILOB2 > stackList.h > pop<T>()
#ifndef STACK_LIST_H
#define STACK_LIST_H
template <typename T>
class Node{
public:
T data;
Node* next;
};
template <typename T>
Node<T> *head = nullptr;
template <typename T>
void push(T newData){
Node<T>* newNode = new Node<T>;
newNode->data= newData;
newNode->next = head<T>;
head<T> = newNode;
}
template <typename T>
T pop(){
if (head<T> == nullptr)
std::cout<< "Stack Overflow \n";
Node<T>* temp = head<T>;
T tempVal = temp->data;
head<T> = head<T>->next;
delete temp;

```
33  T pop(){
53
54  }
55
56
57  template <typename T>
58  void Top(){
59
60
61      if(head<T> == nullptr){
62          std::cout<<"Stack Underflow "<<std::endl;
63          return;
64      }
65
66
67      std::cout<<"Top of Stack: " <<head<T>->data <<std::endl;
68  }
69
70
71  template<typename T>
72  bool isEmpty(){
73      return head <T> == nullptr;
74  }
75
76
77
78
79  template<typename T>
80  void displayAll(){
81
82
83      if(head<T> == nullptr){
84
85
86          std::cout<<"The Stack is Empty\n";
87          return;
88      }
89
90
91      std::cout<<" Displaying the Stack \n";
92      Node<T>* currentNode = head<T>;
93
94
95      while(currentNode != nullptr){
96
97
98          std::cout<< currentNode->data << std:: endl;
```

```
78
79  template<typename T>
80  void displayAll(){
81
82
83      if(head<T> == nullptr){
84
85
86          std::cout<<"The Stack is Empty\n";
87          return;
88      }
89
90
91      std::cout<<" Displaying the Stack \n";
92      Node<T>* currentNode = head<T>;
93
94
95      while(currentNode != nullptr){
96
97
98          std::cout<< currentNode->data << std:: endl;
99          currentNode = currentNode-> next;
100
101     }
102     std::cout << "\n";
103 }
104
105 #endif
106
```

Output of the Code:

```
                                                                    > cd "c:
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
 Displaying the Stack
e
p
c

Top of Stack: e

 ------ Popping all the elements ------
Popped: e
Popped: p
Popped: c

 ----- Testing the Underflow -----
Stack Overflow
PS C:\Users\TIPQC\OneDrive\Documents\CPE 010 - CPE21S1\HOA4P1\ILOB2>
```

Conclusion:

- From the output, I observed that the program correctly performed the basic stack operations. The elements c, p, and e were pushed into the stack, displayed in Last-In, First-Out order, and popped one by one starting from the top element. The program also tested stack overflow, showing how the stack handles overflow

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

4. SUPPLEMENTARY ACTIVITY

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

- a. Stack using Arrays
- b. Stack using Linked Lists
- c. (Optional) Stack using C++ STL

Problem Definition:

Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {,
or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line

Code:

main.cpp

main.cpp > isBalanced(std::string)

```cpp
#include <iostream>
#include <string>

const int maxCap = 100;
char stack[maxCap];
int top = -1;

void push(char data);
char pop();
void Top();
bool isEmpty();
bool isFull();
bool isBalanced(std::string expression);
void displayStack();

int main() {

    std::string expression;
    char choice;

    do {
        std::cout << "Enter expression: ";
        std::getline(std::cin, expression);

        if (isBalanced(expression))
            std::cout << "\nExpression is Balanced." << std::endl;
        else
            std::cout << "\nExpression is NOT Balanced." << std::endl;
```

```cpp
int main() {
    do {
        std::cin.ignore();
    } while (choice == 'Y' || choice == 'y');

    std::cout << "\nProgram Ended." << std::endl;

    return 0;
}

// Check if stack is empty
bool isEmpty() {
    return top == -1;
}

// Check if stack is full
bool isFull() {
    return top == maxCap - 1;
}

// Push
void push(char data) {
    if (isFull()) {
        std::cout << "Stack Overflow!" << std::endl;
        return;
    }

    stack[++top] = data;
```

main.cpp > isBalanced(std::string)

```
51  void push(char data) {
56
57      stack[++top] = data;
58
59      std::cout << "\nPush: " << data << std::endl;
60      displayStack();
61  }
62
63  // Pop
64  char pop() {
65      if (isEmpty()) {
66          return '\0';
67      }
68
69      char temp = stack[top--];
70
71      std::cout << "\nPop: " << temp << std::endl;
72      displayStack();
73
74      return temp;
75  }
76
77  // Display Top
78  void Top() {
79      if (isEmpty()) {
80          std::cout << "Top: Stack is Empty" << std::endl;
81          return;
82      }
```

main.cpp > isBalanced(std::string)

```
78  void Top() {
79      if (isEmpty()) {
82      }
83
84      std::cout << "Top Element: " << stack[top] << std::endl;
85  }
86
87  // Display Stack
88  void displayStack() {
89
90      if (isEmpty()) {
91          std::cout << "Stack: Empty" << std::endl;
92          return;
93      }
94
95      std::cout << "Stack: ";
96
97      for (int i = top; i >= 0; i--) {
98          std::cout << stack[i] << " ";
99      }
100
101     std::cout << std::endl;
102
103     Top();
104 }
105
106 // Balanced Symbols
107 bool isBalanced(std::string expression) {
```

main.cpp > isBalanced(std::string)

```
112
113         char ch = expression[i];
114
115         // Opening Symbols
116         if (ch == '(' || ch == '[' || ch == '{') {
117             push(ch);
118         }
119
120         // Closing Symbols
121         else if (ch == ')' || ch == ']' || ch == '}') {
122
123             if (isEmpty()) {
124                 return false;
125             }
126
127             char open = pop();
128
129             if ((ch == ')' && open != '(') ||
130                 (ch == ']' && open != '[') ||
131                 (ch == '}' && open != '{')) {
132                 return false;
133             }
134         }
135     }
136
137     return isEmpty();
138 }
```

Output:

```
Enter expression: (A+B)+(C-D)

Push: (
Stack: (
Top Element: (

Pop: (
Stack: Empty

Push: (
Stack: (
Top Element: (

Pop: (
Stack: Empty

Expression is Balanced.

Do you want to check another expression? (Y/N): y
Enter expression: ((A+B)+(C-D)

Push: (
Stack: (
Top Element: (

Push: (
Stack: ( (
Top Element: (

Pop: (
Stack: (
Top Element: (

Push: (
Stack: ( (
Top Element: (
```

```
Stack: ( (
Top Element: (

Pop: (
Stack: (
Top Element: (

Expression is NOT Balanced.

Do you want to check another expression? (Y/N): y
Enter expression: ((A+B)+[C-D])

Push: (
Stack: (
Top Element: (

Push: (
Stack: ( (
Top Element: (

Pop: (
Stack: (
Top Element: (

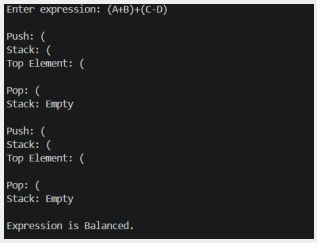
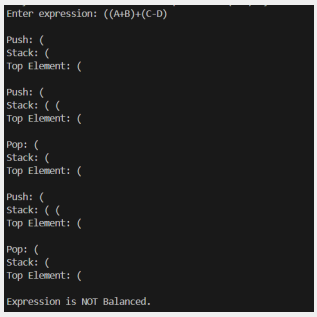
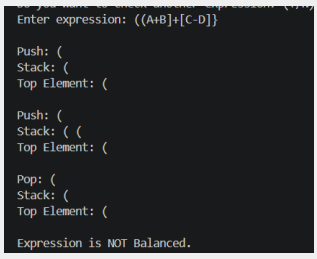
Push: [
Stack: [ (
Top Element: [

Pop: [
Stack: (
Top Element: (

Pop: (
Stack: Empty

Expression is Balanced.
```

SELF CHECKING:

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		All opening parentheses have matching closing parentheses in the correct order, so the expression is balanced.
((A+B)+(C-D)	N		The Parentheses is not equal to another so the one parentheses remains in the stack after scanning the entire expression, making it unbalanced
((A+B)+[C-D])	Y		The parentheses and brackets are properly matched and correctly nested, so the expression is balanced.
((A+B]+[C-D]	N		The closing symbols do not match their corresponding opening symbols closes and closes, so the expression is not balanced

Tools Analysis
How do the different internal representations affect the implementation and usage of the stack?

- During this activity, I learned that the way a stack is implemented can affect how it stores data, but the result is still the same. The array implementation is easier to understand but has a fixed size, while the linked list implementation is more flexible because it can grow as needed. This activity helped me better understand how stacks are used to check if symbols in an expression are balanced.

5. Conclusion / Learning

Summary of lessons learned: - In this activity I learned how stacks work using arrays and linked lists . I also learned how the push, pop, top and display operations are used to solve real problems such as checking whether an expression has balanced symbols. This activity helped me understand the importance of the LIFO principle in stack operations.
Analysis of the procedure: - The steps to follow in coding were simple, but when applying the code you had to be careful with the syntax and the logic. I made some mistakes while coding like unmatched braces, wrong function declarations, but correcting them helped me to learn the program better.
Analysis of the supplementary activity: - The supplementary activity allowed me to use what I had learned about stacks in a real-life scenario. I played with different expressions and understood how the stack keeps the opening symbols and pops them when it encounters the closing ones.
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement? - I think I did better than I expected because I was able to complete the program after correcting several coding errors. Although I still need more and more practice in debugging and writing programs independently, this activity increased my confidence in using stacks and understanding their applications.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.